

Computer Science 3270

Project 2

C++ Programming Project

Paintbots

Introduction

Your task is to create a game that lets players play Paintbots. This game is played on a 15-by-15 square board. Players create robots that maneuver across the board to paint the squares of the board. The game rules are presented below. As part of the assignment, you will not only create the infrastructure for gameplay, but you will also be required to create example robots that play the game. There may be a tournament at the end of the semester where you can enter your robot players to play against each other.

The Paintbot game is played on a 15 by 15 grid on which the two robots reside and compete. Only one robot may occupy a square at a time. (If two robots try to occupy the same square at the same time, they crash. There is more about this later.) Robots may turn 90 degrees left, 90 degrees right, move forward one square, or stay in the same spot. In addition, on any of these moves, the robot may also fire a blob of paint. Each turn, both robots perform their actions at the same time. Thus, each turn of the game, the robots perform the following actions in the order of

1. Both robots may perform a move operation
2. Both robots may fire a paint blob in the direction the robot is facing
3. Both robots may request scan information for the next turn

At the beginning of the game, all squares on the board are painted white (a neutral color) and each robot is randomly assigned a paint color of red or blue. The game lasts for 300 moves, and both robots move simultaneously. During each turn or move of the simulation, the robot may make a move, given by its control algorithm. If the robot moves out of a square, the square that it leaves is turned the robot's color. The goal of the game is to have more squares on the board painted your color than your robot opponent's color. This is accomplished by moving around the board and painting the squares. Thus, it is good to paint over squares colored in your opponent's color, as it adds a square that is your color and takes one away from your opponent. There are, of course, some wrinkles.

1. A robot may shoot a blob of paint, and if it hits the opponent's robot, that robot will paint your color instead of its own when it moves. This will last for a specified number of moves defined by the configuration file. The default value is 20.

2. A robot has only a limited number of paint blobs that it may shoot. This is defined by the configuration file. The default number is 30.
3. If a robot runs into a wall, the game will end, and that robot loses automatically.
4. If a robot runs into the other robot, the game will end immediately, and the robot with the most squares painted its color at that point wins.
5. The game board contains squares that have rocks in them. Neither robot may enter these squares, and running into a square with rocks will terminate the game with a loss for the player that runs into the rock. (Just like running into a wall.) The initial number of rock squares is between 10 and 20, chosen randomly. The configuration file specifies these upper and lower bounds.
6. A robot cannot fire a paint blob through a rock.
7. The game board contains squares that may have fog in them. A robot may shoot through fog. Fog squares can randomly move during the game. The initial number of fog squares is between 5 and 10, chosen randomly. The configuration file specifies these upper and lower bounds. Fog squares restrict the view of the robot for a local scan. See below.
8. Each time a robot's control algorithm decides on the next move, it has access to its current surroundings in the form of a 5 by 5 square, with the robot at the center. The direction it is facing is always considered "up." This is called a short-range or local scan.
9. If a robot is in a Fog square, the 5 by 5 square returned by the local scan consists of all empty white squares except for the center square, which will indicate a fog square.
10. A robot's control algorithm may also receive a limited number of long-range scans which gives the contents of the entire 15 by 15 board. A long-range scan is available on any move. The number of long-range scans is limited and is specified by the configuration file. The default number is 30.
11. At the beginning of the simulation, rocks are placed randomly on the board and the two robots in competition are placed at a random position on the board. Fog squares are also placed at random on the board. Robots are placed so that they start in an empty square (not fog or rock).
12. Fog may hide a rock as both may occupy a square at the same time.

Part a

The purpose of this part of the project is to setup basic simulation classes for playing the game.

1. Create a class named BoardSquare using the files BoardSquare.h and BoardSquare.cpp. This is a super class of two other classes named InternalBoardSquare and ExternalBoardSquare. Create the four files for these two classes: InternalBoardSquare.h, InternalBoardSquare.cpp, ExternalBoardSquare.h and ExternalBoardSquare.cpp. You should have 6 files created after this. The BoardSquare class must minimally contain the following definitions. Note that you will create the SquareInfo and RobotInfo classes in (2) below.

```
class BoardSquare
{
public:
    virtual SquareInfo getSquareInfo() const = 0;

    virtual RobotInfo getRobotInfo() const = 0;
};
```

Note that the above prototypes are semantically similar to abstract methods in Java. In other words, the two methods must be implemented in a subclass.

2. Next, create a SquareInfo.h header file that contains the following enumerated types. Note that SquareInfo and RobotInfo are used to pass information to the opposing Robot. Do not add extra information into these structures that is accessible and could be used by your opponent for an advantage. If you need information such as the robot's direction to create the simulation, it is best to do that in the InternalBoardSquare class.

```
enum PBSquareContent
{
    EMPTY,
    ROCK,
    FOG,
    ROCKFOG,
    WALL
};

enum PBColor
{
    RED,
    BLUE,
    WHITE,
};
```

Also include a SquareInfo class that contains 3 public member variables as follows. (This is essentially a struct type for storing information about a square. You are free to

create an implementation file for this class if you find it useful.)

```
PBSquareContent content;  
PBColor color;  
RobotInfo robot;
```

Finally, create a RobotInfo.h header file that contains information about a robot. It contains the enumerated type

```
enum PBDirection  
{  
    NORTH,    // UP  
    EAST,     // RIGHT  
    SOUTH,    // DOWN  
    WEST,     // LEFT  
};
```

And the member variables

```
bool robotPresent;  
PBColor color; // Robot Color
```

3. Fill in the InternalBoardSquare header file and ExternalBoardSquare header file so that they are subclasses of BoardSquare. An ExternalBoardSquare class, along with scan information, is given to the Robots to compute their next move. An InternalBoardSquare object can be used by the simulation and may contain extra information. Add a constructor to the ExternalBoardSquare class that takes as a parameter an InternalBoardSquare, but only copies information that robots are allowed to use.

```
ExternalBoardSquare(const InternalBoardSquare& internalSquare);
```

You might consider protected member variables in the BoardSquare class to avoid code duplication.

4. Note that (4) implies that this information must also be stored in the InternalBoardSquare, so make the appropriate changes to InternalBoardSquare. You may need to add additional member variables, or replacement member variables of a different type later to support your design choices. For example, it may be useful to store a pointer to the red robot agent if the red robot is present in the square instead of a Boolean variable that indicates the red robot is present in the square.
5. Create a class called Config using Config.h and Config.cpp. This class is used to parse a configuration file. The default constructor creates an object that represents the default configuration. A specific constructor that takes a string parameter that is a path to a

configuration file. The format of the configuration file is defined by the following example.

Lines beginning with the first character of “#” are ignored.

```
# Upper and lowercase characters are not unique.  e.g. HIT_DURATION is the
# same as hit_Duration, both are valid.
#
# Blank lines and lines that contain white-space characters are ignored.
#
# Lines atarting with # are ignored.
#
# The file consists of lines of the form keyword = value.
#
# Keywords and their values can come in any order.
#
# Any missing configuration parameters in the file receive the default value.

# See item 1 in game rules
HIT_DURATION = 28

# See item 2 in game rules
PAINTBLOB_LIMIT = 30

# See item 5 in game rules
ROCK_LOWER_BOUND = 10
ROCK_UPPER_BOUND = 12

# See item 7 in game rules
FOG_LOWER_BOUND = 2
FOG_UPPER_BOUND = 8

@ See item 10 in game rules
LONG_RANGE_LIMIT = 10
```

The constructor must throw an appropriate error object if the file does not exist or the file is not in the correct format. Be sure to look at the string type for useful methods.

6. Add the following functions to the Config class to retrieve configuration information.

```
int getHitDuration();  
int getPaintBlobLimit();  
int getRockLowerBound();  
int getRockUpperBound();  
int getFogLowerBound();  
int getFogUpperBound();  
int getLongRangeLimit();
```

Be sure when you implement these that you declare the function so that changes cannot happen to member variables.

7. Create a file named ConfigTest.cpp that contains a main method. The goal of this file is to test if the Config class works properly. Your file should consist of several functions that test various possible faults in your Config class code. The main method in this file calls these functions sequentially and outputs the result for each function. For example, sample output might look like:

Config class unit tests

```
checking default constructor: PASS  
checking bad file name: PASS  
checking valid config file, all parameters: PASS  
checking valid config file, missing parameters: PASS  
checking invalid config file: FAIL  
    No equals in parameter line 5, but no error thrown
```

Etc...

Make sure you write at least 10 unique tests for this class.

8. Create a makefile to compile all of the files above. Also include a target named “testconfig” that creates the executable named “testconfig” to test the Config class.
9. Create and use a new repository on **git.las.iastate.edu** called **coms3270P2**. When completed, create a tag named **partacomplete** for your working and documented submission.

Part b

Just like in the real world (or because the instructors want to give you more in-depth practice, provide you with more information, and help you better understand the material for the exams), there is significant refactoring in Part b. It is strongly suggested that you make a git branch for Part a, and make sure your tag points to the branched version. We will still grade Part a based on the Part a instructions.

For this part of the assignment, we aim to create a complete GameBoard object that represents any game state. This object will not be responsible for controlling the overall game, but it will determine if the move is legal based on a configurable rule set. Note that you will use the strategy pattern among others to accomplish this.

1. Before you start changing your project, make sure you have tagged Part a on `git.las.iastate.edu` as in Item 10 in Part a. Again, make a branch for Part a as this could save you much headaches in the future.
2. Delete from your project all header and cpp files except for the `config.h` and `config.cpp`. Did we mention that you may want to make a branch for part a?
3. Create a class named Util using filenames `Util.h` and `Util.cpp`. In this class define the Color enumerated class with these colors: Red, Blue, White, None. Note the uppercase first characters, as case is important for grading. Also in the Util files include a structure that defines the location of a square on the board. Because it is a struct types, all members are public by default, and looks like this:

```
struct Location
{
    int row;
    int col;

    // Other member methods go here as described below.
};
```

The other member functions required in this class (struct) are:

- Default constructor that creates the 0, 0 location.
- A Specific constructor that takes row and column parameters in that order and creates the specified location.
- An overladed “+” operator that adds to locations together in the same way to vectors are added together, with the result residing in the first operand.
- An overloaded “-” operator that subtracts the second (other location) from the first with result residing in the second operand.
- An overloaded “==” operator that tests if two locations are the same.
- An overloaded “!=” operator that tests if two locations are not the same.

Place the implementations of these operators in util.cpp.

Also in the Util files, create a class called Direction

```
struct Direction
{
    int dRow;
    int dCol;

    static const Direction North;
    static const Direction East;
    static const Direction South;
    static const Direction West;

    // Other method prototypes go here
};
```

Make sure you initialize the static directions correctly in the Util.cpp file, otherwise you will have unresolved link errors. (The North direction is dRow = -1 and dCol = 0 for up. The other methods required in this structure (class) are: a default constructor that initialize the direction to North, and a copy constructor. (Note that the default copy constructor would work here, but its good practice to write it.) Also include the following methods:

```
Direction RotateLeft90() const; // Return the direction that is the direction
                                to the left. Left of North is West.

Direction RotateRight90() const; // Return the Direction that is the direction
                                to the right. Right of North is East.

Direction Rotate180() const;    // Return the Direction that is the direction
                                opposite. Opposite of North is South.
```

Place all these implementation in Util.cpp.

Note that there is a constructor defined for Location and there is no constructor defined for Direction. This is to illustrate two different ways in which they are initialized. Typically you would probably not include a constructor with data that is this simple.

4. Although you would likely do this differently do to its simplicity and the fact that it does not need to be extensible, we will utilize a Strategy-like pattern to keep track of the contents of each square on the board. To start, we create several classes

The first priority is to (if necessary) refactor board squares so that InternalBoardSquare and ExternalBoardSquare can share common properties. To do this, we will create the

Content.h and Content.cpp files and place the several classes that represent a square's content, analogous to the student behavior classes we did in lecture. The Content.h files contains the following:

```
#pragma once

#include "Util.h"

class Content
{
public:
    virtual bool isPresent() const = 0;
};

class NotPresent : public Content
{
public:
    bool isPresent() const;
};

class RockContent : public Content
{
public:
    bool isPresent() const;
};

class FogContent : public Content
{
public:
    bool isPresent() const;
};

class WallContent : public Content
{
public:
    bool isPresent() const;
};

class SquareColorContent : public Content
{
private:
    Color color;
public:
    SquareColorContent(Color c);
    bool isPresent() const;
    Color getColor() const;
    void setColor(Color c);
};

class RobotContent : public Content
{
private:
    Color color;
    Direction direction;
    Color paintColor;
public:
    RobotContent(Color c, Direction d, Color p);
    bool isPresent() const;
    Color getColor() const;
    Direction getDirection() const;
    Color getPaintColor() const;
};
```

The implantation in Content.cpp should be straight-forward.

5. We can now define the Square class that uses the content types defined above. The Square.h file contains the following:

```
#pragma once

#include "Util.h"
#include "Content.h"
#include "Square.h"

class Square
{
public:
    Content *Rock = &NotPresentInstance;
    Content *Fog = &NotPresentInstance;
    Content *SquareColor = &NotPresentInstance;
    Content *Robot = &NotPresentInstance;
    Content *Wall = &NotPresentInstance;

    static NotPresent NotPresentInstance;
    static RockContent RockInstance;
    static FogContent FogInstance;
    static WallContent WallInstance;

    Square();

};

class ObscuredSquare : public Square
{
public:
    ObscuredSquare();

};

class AgentSquare : public Square
{
private:
    SquareColorContent agentSquareColor = Color::None

public:
    AgentSquare(Color squareColor, bool rockPresent, bool fogPresent, bool robotPresent,
                Color robotColor);
    AgentSquare(InternalSquare s);
};

class InternalSquare : public Square
{
private:
    SquareColorContent internalSquareColor = None;
    RobotContent RobotInstance = RobotContent(Color::None, Direction{ 0,1 }, Color::None);

public:
    InternalSquare();

};
```

The file also contains three subclasses for the different types of squares. Obscured squares have no information about them and are used to with external robot agents when

a square is not seeable due to fog. When passed to external code, there should be no way the external agent can use casting or other methods to determine what is in the square. Agent squares contain only information that is allowed for use by agents to determine their next move. Finally the internal square is used in the game execution. Additional member variables and methods can be added to this class to support the game execution.

Here are some useful notes about these classes. Rock, Fog, SquareColor, Wall, and Robot are all pointers to Content types. These form the “behaviors” we talked about in lecture for the strategy pattern. We also define some static versions of these since they are Boolean behaviors, not mutable, and only the isPresent virtual function required for information. For example, on construction of this object, all the behaviors point to the NotPresentInstance static object. These static objects must be initialized in the cpp implementation file.

The implementation of these classes is pretty minimal. The default destructors, copy constructors, and assignment equals operator will all suffice. The exception is if you add non trivial items to the InternalSquare class, you may need to write them. Also note how a Direction structure is created and passed to the RobotContent constructor.

Lastly, you are to create a GameBoard class and use the filename GameBoard.h and GameBoard.cpp. The GameBoard class must implement the Singleton pattern. Because this class will facilitate robot moves, include the following RobotMoveRequest header file in your code. Be sure to copy this exactly as it forms the interface between the GameBoard and external agents.

```
enum RobotMove {TurnLeft, TurnRight, MoveForward, NoMove};

struct RobotMoveRequest
{
    RobotMove move;
    bool fire;
};
```

Now create the GameBoard.h and GameBoard.cpp files. In the private section of the class include the following lines of code.

```
GameBoard(const GameBoard&) = delete;
GameBoard& operator=(const GameBoard&) = delete;
GameBoard();
```

The first two lines of code remove the possibility of create a copy of an object of this class, since it implements the singleton pattern. The last line creates a default private constructor that can only be called from within this class. You may add private member variables and methods to this class if needed. The following public prototype methods must be in your header file GameBoard.h.

```

static GameBoard& getInstance(string password);

void resetBoard();

void resetBoard(Config c);

Square& getSquareContent(const Location& loc);

bool moveRobot(RobotMoveRequest moveRequest);

bool paintBlobHit(RobotMoveRequest moveRequest);

void setSquareColor(const Location& loc, Color color);

const RobotContent& getRobotContent(Color robotColor);

int blueScore() const;
int redScore() const;
int turnNumber() const;

AgentSquare** getLongRangeScan(Color robotColor) const;

AgentSquare** getShortRangeScan(Color robotColor) const;

void testPrintBoard();
void testPrintSquare(Location loc);

```

The `getInstance` method requires a password which if correct, returns a reference to the one and only instance of the `GameBoard` object. Otherwise, it should throw the exception: `std::invalid_argument("Invalid password")` object, which is in the `#include <stdexcept>` file. For your code, the password should be “xyzyzy”

`resetBoard` methods reset the board to a default configuration (no parameter) or to the configuration specified by the `Config` object parameter.

The `getSquareContent` returns a reference to the `Square` at the specified location. This method is likely not used often since in this class the board can be accessed directly. If the specified location is not in the range of a legal square, an exception may be thrown.

The `moveRobot` method takes a `MoveRequest` object and performs the action, reflecting any robot moves on the board. (Firing is not considered a robot move.) The method returns true if it was able to make the move.

The `paintBlobHit` returns true if the robot would hit the other robot with a paint blob if it fired.

The `setSquareColor` method sets the color of the square on the board at the specified location.

The `getRobotContent` returns a reference to a `RobotContent` object for the specified robot by color.

`blueScore` and `redScore` returns the number of square on the board of that color.

`getLongRangeScan` returns an entire board in terms of `AgentSquare` objects. Note that since this will be going to robot agents, a new array is created with the copy constructor

invoked. The long range scan is always oriented with north up for the array. A different object in Part c will handle if the requesting robot is in Fog.

getShortRangeScan returns the 9 squares around a location. This also must be copied to an AgentSquare type, however, the scan is oriented so that up is the robot's forward direction.

The two print methods are used to print a board using text to the screen. This is useful for testing. testPrintBoard prints the entire board. (example below) and the testPrintSquare prints the contents of the specified square, where each square is represented by three characters. The first character is the square color (W, B, R), the second character is the obstructions (F for fog, R for Rock, and X for fog and rock) and the final character is for red or blue robot (R, B). In all cases, a - character means that that attribute is not there. For example: W-- is a plain white square, WR- is a rock square, and B-B is a square painted blue with the blue robot in it.

An example output from testPrintBoard:

```
W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W--
W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W--
W-- WR- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W--
W-- W-- W-- W-B W-- W-- W-- W-- R-- R-- R-- W-- W-- W-- W--
W-- W-- W-- B-- W-- W-- W-- W-- R-- W-- W-- W-- W-- W-- W--
W-- W-- W-- B-- W-- WR- W-- W-- W-R W-- W-- W-- W-- W-- W--
W-- W-- W-- B-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W--
W-- W-- W-- B-- W-- W-- W-- W-- W-- W-- W-- WF- W-- W-- W--
W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W--
W-- W-- W-- W-- W-- W-- WR- W-- W-- W-- W-- W-- WX- W-- W--
W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W--
W-- W-- W-- W-- WF- W-- W-- W-- W-- WF- W-- W-- W-- W-- W--
W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W--
W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W--
W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W-- W--
```

6. Finally, create the file TestGameBoard.cpp and place a main function in the file that loads a configuration file given as the first parameter in the command line, creates a GameBoard object based on that configuration, adds a blue and red robot to an empty square on the board, and displays the board.
7. Update makefile and documentation appropriately and tag this part **partbcomplete**.

Note that in some cases you may need to downcast objects. C++ has two other ways to do this besides a standard cast that has some advantages. The first is called a static cast that checks if the cast could be made at compile time and otherwise causes an error in the compilation. The second is called a dynamic cast and checks if the cast to a pointer type can be made at runtime, and if not, returns null. More information can be found at:

https://www.geeksforgeeks.org/cpp/static_cast-in-cpp/ and
https://www.geeksforgeeks.org/cpp/dynamic_cast-in-cpp/

Part c

For this part of the project, we will complete the simulation game by designing and coding a class to perform the game logic and build robot agents to play in the simulation.

1. First create a header file named **RobotAgent.h** that contains an interface (class with all pure virtual functions) for the class **RobotAgent**. The interface contains exactly these four functions.

```
virtual string getAgentName() = 0;
virtual string getAgentCreator() = 0;
virtual void getAgentMove(RobotMoveRequest& rmr, GameBoard::SROW * srs, GameBoard::ROW * lrs) = 0;
virtual void setRobotColor(Color c) = 0;
virtual RobotAgent* clone() = 0;
```

2. Create a class named **RobotAgentRoster** that keeps a list of **RobotAgents**. Define this class in **RobotAgentRoster.h** and **RobotAgentRoster.cpp** appropriately. This class includes a default constructor that creates a roster of robot agents that at a minimum includes the robot agents that you are required to write for this assignment. Also add an **add(RobotAgent *)** that adds a pointer to a robot agent to the end of the list. Include the overloaded operator **[]** that returns a pointer to the i^{th} agent starting at index 0. Also include a different overloaded **[]** operator that takes a string argument and returns the robot agent pointer with the matching name.
3. Create two **RobotAgents** that implement the **RobotAgent** interface. These two classes should be called **LazyAgent** and **RandomAgent**. (with name fields that match) The lazy robot agent does not move from the initial square and fires a paint blob each turn in a random direction. The random robot agent moves in a random direction to a legal square (non rock or wall or other robot) and never fires. These classes are defined in the file **TestAgents.h** and implemented in the file **TestAgents.cpp**. Note that the clone method returns a new object that is a deep copy of the robot agents.
4. Finally, create a class named **GameLogic** in **GameLogic.h** and **GameLogic.cpp** that is used to play a game. This class contains the following requirements.
 - a. Implements the singleton pattern with the same password as the **GameBoard**, eg, a **getInstance(string password)** method.
 - b. The class contains a method named **resetGame()** that initializes a game with the default configuration. This method is overloaded **resetGame(Config c)** that initializes a game with the specified configuration object.

- c. A method with **playGame(string redname, string bluename);**
The two parameters are used to clone the two specified robot agents from a RobotAgentRoster object. The method then simulates the game according to the rules at the beginning of this document until it ends and prints a winner. After each turn (both robots make a move), a new board is printed in the format specified in part b, but modified per below.
 - d. Update the testPrintBoard method so that each square now contains 4 characters. The last character indicates the direction of the robot if a robot is present. The valid characters are **n, e, s, w**, and **-**. The later - character is printed if a robot is not present in the square.
 - e. You probably will want to add additional functions and variables to this class to make it more readable. Please do so appropriately.
- 5. Create a main method in the the file PaintBot.cpp that takes three command line parameters. The first two parameters are the names of the red and blue robot, respectively. If present the third parameter is the path to a config file to use. If the third parameter is not present, then the default configuration is used.
 - 6. Create a robot agent that at least 8 times out of 10 will beat the random robot. Place this robot agent in a class named AntiRandom and agent name BeatRandom.
 - 7. Create a robot agent that tries to follow the opposing robot by following its trail of paint. When appropriate, the robot should make long range scans and fire a paint blob at appropriate times.
 - 8. Update the makefile appropriately so that by just typing make, all the files are compiled and the executable file is created named **paintbots** that is the main method for playing the game described above.